

Comprehensive Analysis of the CWIIF Algorithm: Limitations, Computational Trade-offs, and Architecting an Optimized Implementation

The intricate topology of complex systems—ranging from intermodal transportation grids and financial markets to biological ecosystems and social communication platforms—demands robust analytical frameworks that can navigate multiple, overlapping dimensions of connectivity. Traditional network science has historically relied on single-layer topologies, which often fail to encapsulate the heterogeneous interactions inherent in modern complex systems. Multilayer networks offer a mathematically rigorous representation of these systems, mapping distinct modes of interaction to separate architectural layers. Within this domain, identifying the most influential nodes—the influence maximization problem—is a paramount objective for applications spanning viral marketing, epidemic control, and critical infrastructure defense¹.

The Coupling Weighted Intra-layer and Inter-layer Influence Factors (CWIIF) algorithm emerged as a novel methodology designed to pinpoint these influential nodes by balancing a node's local structural importance with the macro-level significance of the layers it inhabits¹. Rather than naively collapsing multilayer structures into a single aggregated adjacency matrix, CWIIF computes layer-specific weights based on node activity, transmission path efficiency, and inter-layer connectivity¹. It subsequently pairs these weights with intra-layer centrality vectors to generate a comprehensive ranking of node influence¹. While initial empirical testing utilizing Susceptible-Infected (SI) and Susceptible-Infected-Recovered (SIR) propagation models demonstrated promising results across benchmark datasets such as the Vickers-Chan social network, the London transport multiplex, and the *Drosophila* genetic network, the algorithm exhibits profound structural, computational, and topological vulnerabilities¹.

Architecting an improved, production-ready iteration of the CWIIF framework necessitates an exhaustive deconstruction of its baseline limitations. This report provides a deep, technical analysis of the algorithm's downsides, computational trade-offs, and graph-theoretical constraints. Furthermore, it details a comprehensive blueprint for overriding these limitations through the integration of randomized approximation algorithms, asynchronous parallel processing, topological tensor formulations, and evolutionary optimization models.

Mathematical Foundations of the Baseline

Architecture

To rigorously evaluate the computational and topological limitations of the CWIIF algorithm, one must first dissect its underlying mathematical formalization. The algorithm operates on a

multiplex network denoted as $G = (L, V, E)$, where L represents the set of layers, V constitutes the set of nodes, and E signifies the collection of intra-layer edges¹. The multiplex assumption asserts that $V^{[1]} = V^{[2]} = \dots = V^{[L]}$, indicating that every entity exists concurrently across all network layers, although the edge configurations $E^{[\alpha]}$ may vary drastically from one layer to the next¹.

The baseline CWIIF operates through a sequential three-phase pipeline intended to map multi-dimensional topological data into a singular scalar ranking¹.

The first phase focuses on computing a macro-level weight for each layer, denoted as $W(\alpha)$, by synthesizing three normalized environmental factors¹. The Normalized Active Node (NAN) metric quantifies the density of highly connected entities within a layer. An active node is

strictly defined as an entity i residing in layer α whose local degree exceeds the average

degree of that entire layer, expressed as $K_i^{[\alpha]} > \langle K^{[\alpha]} \rangle$ ¹. The NAN value for layer α is the ratio of its active nodes to the total active nodes across all layers within the network. Following this, the algorithm evaluates transmission efficiency via the Normalized Active Path (NAP) metric. The active path calculates the exact shortest topological distance between all

non-isolated node pairs i and j , denoted as $d_{ij}^{[\alpha]}$. The average of these minimal paths defines the layer's raw transmission capacity, which is subsequently normalized against the overarching network structure¹. The final component of the layer weight is the Normalized Inter-layer Communication Intersection (NICI). This metric attempts to capture cross-layer topological

similarities by computing the Jaccard similarity coefficient of the edge sets between layer α and all other layers β ¹. The intersection of these edge sets is divided by their union, creating a

raw overlap score that is then normalized. The final layer weight $W(\alpha)$ is derived by executing a uniform, unweighted summation of these three normalized components.

The second phase transitions from macro-level layer assessment to micro-level node evaluation, computing an intra-layer influence vector for each entity. The algorithm achieves

this by isolating each layer and calculating a composite centrality score, $KB_i^{[\alpha]}$, which acts as a linear combination of two normalized topological metrics: K-shell decomposition (KS) and betweenness centrality (BC)¹. The K-shell metric measures a node's topological coreness by iteratively peeling away peripheral nodes with the lowest degrees, thereby assigning higher values to nodes deeply embedded in dense subgraphs⁶. Conversely, betweenness centrality measures the global bridging capacity of a node by quantifying the precise fraction of all possible shortest paths that intersect it⁸. By fusing a local density metric with a global routing metric, the algorithm aims to capture a holistic profile of the node's influence within that

specific layer.

The third and final phase executes a dimensionality reduction, compressing the

multi-dimensional vectors into a final scalar metric. The Total Influence (TI_i) of a given node

i is mathematically defined as the dot product of its intra-layer influence vector and the previously computed layer weights, effectively scaling the node's localized importance by the overall significance of the layers it inhabits¹. This scalar value dictates the final ordinal ranking of the nodes, which is utilized for downstream influence maximization tasks.

Computational Complexity and Temporal Trade-offs

The most prominent barrier to deploying the baseline CWIIF algorithm in production environments or applying it to massive, real-world graphs is its catastrophic computational complexity. The theoretical time bounds established in the baseline framework render it entirely intractable for contemporary network science applications, trading all practical scalability for mathematical exactness.

The Exponential Bottleneck of Exact Shortest Paths

The algorithm requires the computation of All-Pairs Shortest Paths (APSP) at two distinct junctures in its execution lifecycle. First, APSP is strictly required to calculate the average active

path length ($d^{[\alpha]}$) for the NAP factor¹. Second, the computation of exact betweenness

centrality ($BC_i^{[\alpha]}$) demands the identification of every shortest path traversing the network to determine the routing fractions for every individual node¹.

The original CWIIF methodology explicitly dictates the use of the Floyd-Warshall algorithm to process the shortest paths in the NAP component¹. The Floyd-Warshall algorithm carries an

immutable time complexity of $\mathcal{O}(L \times N^3)$, where L is the number of layers and N is the number of vertices¹. While this approach correctly identifies exact shortest paths in dense graphs, its cubic scaling is fatal for modern networks. For a multiplex network containing a mere 10,000 nodes—a size considered trivially small in contemporary contexts like social media analytics or biological connectomics—the Floyd-Warshall implementation requires on the order of 10^{12} basic operations per layer¹.

Even if the implementation were optimized to replace Floyd-Warshall with Brandes' exact algorithm for betweenness centrality, the computational trade-off remains severely skewed. Brandes' algorithm reduces the APSP complexity by leveraging structural dependencies,

yielding a time bound of $\mathcal{O}(L \times N \times |E|)$ for unweighted graphs and

$\mathcal{O}(L \times (N \times |E| + N^2 \log N))$ for weighted structures¹⁰. While mathematically

superior to cubic bounds, this $\mathcal{O}(NM)$ requirement is still prohibitively slow for scale-free

networks exhibiting the small-world phenomenon, where the number of edges $|E|$ routinely

exceeds 10^7 ¹³. Executing N continuous Breadth-First Search (BFS) or Dijkstra traversals across multiple layers forces excessive cache thrashing and memory bus saturation, preventing the baseline algorithm from scaling beyond highly localized, synthetic datasets.

Algorithmic Component	Baseline Time Complexity	Scaling Constraint	Practical Impact on Massive Networks (N>106)
NAP Calculation (APSP)	$\mathcal{O}(L \times N^3)$ via Floyd-Warshall	Cubic	Completely intractable; execution halts due to integer overflow or thermal throttling.
Exact Betweenness Centrality	$\mathcal{O}(L \times N \times E)$ via Brandes	Quadratic-equivalent	Extremely slow; requires months of dedicated CPU time for a single execution pass.
K-Shell Peeling	$\mathcal{O}(L \times (N + E))$	Linear (Sequential)	Severe memory latency due to serialized data dependencies and cache misses.
Jaccard Intersection (NICI)	$\mathcal{O}(L^2 \times \overline{ E })$	Polynomial in L	Memory intensive; requires massive concurrent memory overhead to hold multiple layer adjacency matrices.

Sequential Dependencies in K-Shell Decomposition

The calculation of the K-shell metric introduces a distinct category of computational limitation. While the theoretical time complexity of K-shell decomposition is bounded by

$\mathcal{O}(L \times (N + |E|))$, which is technically linear relative to the network size, the traditional peeling algorithm is intrinsically sequential¹. The standard implementation iteratively identifies

nodes with the smallest degree, removes them from the graph, and subsequently updates the degrees of all adjacent neighbors⁷.

This serial dependency chain prevents vectorization and creates extreme memory bottlenecks. For large-scale multilayer graphs that cannot comfortably reside within primary memory (RAM), the operating system must page graph partitions to secondary storage¹⁵. Because the deletion of a single high-degree hub can trigger cascading degree updates across disparate memory sectors, the sequential K-shell process generates massive disk fragmentation and continuous I/O thrashing¹⁵. The baseline CWIIF makes no structural allowances for distributed processing, external-memory algorithms, or parallelization for this component, capping its maximum throughput and severely restricting the size of the networks it can analyze¹.

Cross-Layer Intersection Overhead

The calculation of the NICI metric further exacerbates memory and processing overhead. The algorithm utilizes the Jaccard similarity coefficient to quantify the exact intersection of edge sets between all combinations of layers¹. The time complexity for this specific operation is

$\mathcal{O}(L^2 \times |\overline{E}|)$, requiring the algorithm to iterate through all $L(L - 1)/2$ layer pairs¹.

While polynomial in terms of layers, generating union and intersection sets necessitates loading multiple massive adjacency matrices into memory simultaneously. In highly dense networks

with dozens of layers, this $\mathcal{O}(L^2)$ matrix intersection process monopolizes heap memory, causing garbage collection pauses in managed languages or triggering out-of-memory fatal errors in unmanaged environments, thereby severely hindering multi-threaded scaling.

Topological Limitations, Graph Constraints, and Exceptions

Beyond purely computational constraints, the baseline CWIIF algorithm operates under a rigid set of topological assumptions. These structural assumptions artificially narrow the scope of network geometries that the algorithm can process, creating profound analytical blind spots and methodological exceptions when applied to raw empirical data.

The Multiplex Rigidity and the Isolated Node Exception

A fundamental prerequisite of the baseline algorithm is its constraint to strictly multiplex networks¹. Under this definition, every single node must exist concurrently across all network

layers, creating a perfect topological alignment where $V^{[1]} = V^{[2]} = \dots = V^{[L]}$ ¹.

However, generalized multilayer networks in the real world rarely conform to this idealized structure. In a multi-modal transportation network comprising international flights, inter-city trains, and local bus routes, the vast majority of local bus stops do not function as international airport terminals¹.

When the baseline CWIIF algorithm is forced to process a generalized multilayer network, entities that do not participate in specific layers manifest as isolated vertices possessing a degree of zero. The original algorithm attempts to handle this exception by explicitly excluding isolated nodes from the active path length calculations in the NAP metric¹. However, this

programmatic exception introduces a catastrophic statistical bias into the overarching layer evaluation. If a layer contains a massive number of isolated nodes because its interaction

modality only applies to a hyper-specific elite sub-population, the average shortest path $d^{[\alpha]}$ is calculated exclusively over this tiny, densely connected clique. Consequently, the active path length becomes artificially deflated, making the layer appear exponentially more efficient and dense than the overarching network. This mathematical anomaly incorrectly inflates the NAP coefficient, thereby corrupting the layer's macro-weight $W(\alpha)$ and disrupting the entire influence ranking hierarchy.

Incapacity for Weighted and Directed Systems

As explicitly documented in its foundational literature, the baseline CWIIF algorithm is strictly constrained to unweighted and undirected networks¹. This represents a crippling limitation for modeling realistic complex systems where interaction pathways exhibit directionality, friction, and varying capacities.

In the context of directionality, social influence and information propagation are frequently asymmetric phenomena. On platforms like Twitter or academic citation networks, influence flows directionally from a source to a recipient¹⁶. By enforcing an undirected graph topology, CWIIF assumes that all influence is strictly bidirectional. This critical flaw causes the algorithm to dramatically overestimate the global reachability of nodes located in network sinks (nodes that receive many connections but emit very few), assigning them high betweenness centrality scores despite their inability to effectively propagate contagions outward⁹.

Furthermore, the unweighted constraint forces the algorithm to treat all edges as mathematically identical, binary connections (0 or 1)¹. In empirical systems, edges possess highly variable capacities; traffic networks experience distinct travel times, neural networks have varying synaptic strengths, and epidemiological contact networks have varying transmission probabilities¹⁹. Because the baseline CWIIF relies on unweighted Breadth-First Search (BFS) logic to determine active paths and betweenness centrality, the resulting metrics reflect strictly the topological hop count rather than the actual latency, cost, or probability of propagation¹. A path comprising three low-probability edges may be mathematically ranked higher than a path comprising four absolute-certainty edges, leading to severely flawed influence maximization predictions.

The Boundary Problem in Independent Layer Assessment

While the CWIIF framework acknowledges the existence of inter-layer edges $E_i^{[\alpha,\beta]}$ representing the projection of an entity across distinct layers, it structurally ignores them during the second phase of execution¹. The algorithm dictates that intra-layer influence vectors must be calculated completely independently on each layer¹.

By severing all inter-layer connections before computing K-shell and betweenness centrality, the algorithm falls victim to the topological "boundary problem." An entity might act as the exclusive structural bridge linking a massive component in layer A to a massive component in layer B. However, because the intra-layer centrality algorithms are explicitly barred from

'seeing' cross-layer topological connections, this critical bridge node will score extremely poorly when evaluated in isolation³. Consequently, its pivotal bridging capability is entirely overlooked, resulting in a deflated Total Influence (TI_i) score that fails to capture its true macroscopic significance⁸.

Methodological Deficiencies: Weighting Rigidity and Information Loss

The mathematical formulations embedded within the CWIIF architecture rely on arbitrary linear aggregations and scalar mapping techniques. These heuristic shortcuts strip away crucial multidimensional context, representing a profound methodological downside that limits the algorithm's robustness.

The Fallacy of Uniform Linear Combinations

The calculation of the layer-weighted coefficient $W(\alpha)$ is achieved through a naive, unweighted summation: $NAN^{[\alpha]} + NAP^{[\alpha]} + NICI^{[\alpha]}$ ¹. The underlying justification for this approach posits that because the factors evaluate distinct topological dimensions, they should be assigned equal mathematical weight to prevent any single factor from dominating the outcome¹. This represents an arbitrary theoretical assumption that completely ignores the empirical variance of different network domains.

In an epidemiological transmission network, the average shortest path length (NAP) frequently dictates the velocity and saturation of an outbreak far more aggressively than the static structural overlap of communication intersections (NICI)¹⁹. Conversely, in a financial contagion scenario, NICI—which represents the exact overlap of asset classes and counterparty risks across different markets—might be the exclusive driver of systemic vulnerability. By enforcing a strict equal-weighting paradigm, the algorithm homogenizes these disparate phenomena, mathematically forcing all layers into a sub-optimal equilibrium that fails to respect the unique physics of the target system.

This exact vulnerability extends to the computation of intra-layer influence, where the algorithm simply adds normalized variants of K-shell and betweenness centrality:

$KB_i^{[\alpha]} = KS_i^{[\alpha]} + BC_i^{[\alpha]}$ ¹. K-shell is a metric of dense local clustering, whereas betweenness centrality is a metric of global path traversal⁶. By combining these metrics with a rigid 1:1 ratio, the algorithm mathematically dilutes the discriminative power of both, operating under the flawed assumption that a unit increase in core density is always equivalent to a unit increase in global bridging capacity¹⁷. In networks characterized by extreme core-periphery structures or heavy-tailed degree distributions, this rigid linear combination consistently fails to identify nodes that are highly influential purely through bridge mechanics rather than core clustering¹⁷.

Dimensionality Collapse and Sub-Optimal Seed Selection

The terminal stage of the CWIIF algorithm compresses a node's multidimensional influence

vector $KB_i = [KB_i^{[1]}, \dots, KB_i^{[L]}]$ into a single scalar value TI_i via a dot product with the layer weights¹. While scalarization provides a computationally convenient mechanism for generating a flat, sorted list of nodes, it directly triggers catastrophic information loss. When generating seed sets for influence maximization under competitive routing or viral marketing constraints, the spatial diversity of the seed set is critically important²¹. Selecting seeds that exhibit overlapping spheres of influence leads to severe diminishing returns, a phenomenon frequently documented in network science²¹. Two nodes might both achieve a high TI_i scalar score of 0.85. Node A might derive its score by acting as an absolute central hub exclusively in Layer 1, while Node B might derive the exact same score from acting as an absolute central hub in Layer 1 as well. If the algorithm selects both Node A and Node B based purely on their collapsed scalar rankings, it wastes budget by oversaturating Layer 1 while leaving Layers 2 through 5 entirely untargeted. A robust influence maximization algorithm must utilize the full multidimensional context of the network to minimize influence overlap²¹. By compressing the vectors into a scalar prior to the final selection phase, the baseline CWIIF completely blinds downstream applications to the spatial and layered distribution of node influence, inevitably leading to clustered, highly redundant, and sub-optimal seed selections²².

Deficient Baseline Methodology	Architectural Vulnerability	Consequence on Network Analysis
Uniform Factor Weighting ($NAN + NAP + NICI$)	Assumes equal empirical relevance of disparate metrics.	Fails to adapt to specific network physics (e.g., epidemic vs. financial).
Linear Combination of KS and BC	Assumes 1:1 equivalency of local density to global bridging.	Dilutes discriminative power in heavy-tailed scale-free networks.
Scalar Dimensionality Collapse (TI_i)	Erases spatial layer data before seed set selection.	Causes severe influence overlap; selects redundant, clustered seeds.

Architecting CWIIF 2.0: Algorithmic Implementations and Improvements

Implementing an improved, highly optimized version of the CWIIF algorithm—CWIIF

2.0—requires systematic architectural overhauls. These overhauls must aggressively replace exact combinatorial functions with mathematically guaranteed approximation schemes, replace rigid linear heuristics with dynamic data-driven weighting, and generalize the fundamental framework to seamlessly support weighted, directed, and heterogeneous topologies.

1. Resolving the Computational Bottlenecks: Scalable Centrality

To bypass the fatal $\mathcal{O}(N^3)$ computational complexity of the baseline CWIIF, the exact algorithmic calculations for betweenness centrality and active path lengths must be entirely deprecated.

Randomized Approximation for Betweenness Centrality via VC-Dimension

Instead of naively computing the shortest paths between all $\mathcal{O}(N^2)$ node pairs, an optimized implementation must leverage progressive adaptive sampling to estimate betweenness centrality. Frameworks such as the Adaptive Betweenness Centrality Approximation (ABAS) or KADABRA (ADaptive Algorithm for Betweenness via Random Approximation) represent the current state-of-the-art in scalable centrality estimation⁹.

These advanced algorithms reframe the calculation by sampling pairs of source and target nodes uniformly at random, charting the shortest paths specifically between those randomized pivots. The critical innovation is that the required sample size is rigorously governed by Vapnik-Chervonenkis (VC) dimension theory⁸. By applying non-uniform Rademacher averages and VC bounds, the algorithm mathematically guarantees that the maximum deviation

between the approximate betweenness centrality $\tilde{bc}(v)$ and the exact centrality $bc(v)$ is tightly bound by a user-defined additive factor ϵ , with a confidence probability of $1 - \delta$ ⁸.

The sample size required for this guaranteed approximation is dynamically adjusted based on the progressive reduction of variance in the centrality estimates as the algorithm runs⁹. By substituting exact calculations with VC-bounded adaptive sampling, the time complexity plummets from $\mathcal{O}(NM)$ to $\mathcal{O}(\frac{c}{\epsilon^2}(N + M))$, transforming a mathematically intractable operation into a highly scalable, near-linear time task¹³. In the specific context of influence

maximization, where the objective is to determine the absolute ordinal ranking of the top K nodes rather than their fractional centrality scores to the tenth decimal place, VC-dimension adaptive sampling delivers the exact required operational outcome at a fraction of the hardware cost⁸.

Approximating the Normalized Active Path (NAP)

The calculation of the average active path length $d^{[\alpha]}$ must undergo a similar paradigm shift. The use of Floyd-Warshall must be eliminated. CWIIF 2.0 should employ a randomized approximation algorithm specifically designed for distance and closeness metrics. As theoretically proven by Eppstein and Wang, the average distance across a graph exhibiting the

small-world phenomenon can be estimated to within a $(1 + \epsilon)$ factor by executing Single-Source Shortest Path (SSSP) queries from a logarithmically bounded set of randomly chosen pivot nodes¹³.

By strategically selecting $k = \Theta(\frac{\log N}{\epsilon^2})$ pivot vertices and computing Dijkstra's or BFS trees strictly from these pivots, the algorithm completely bypasses the need for all-pairs traversal.

The total computation time to approximate the exact $d^{[\alpha]}$ drops precipitously to $\mathcal{O}(\frac{\log N}{\epsilon^2}(N \log N + |E|))$ for weighted networks, effectively matching near-linear bounds while maintaining high statistical confidence¹³.

Asynchronous and Parallel K-Shell Decomposition

While the K-shell algorithm is fundamentally bounded by $\mathcal{O}(N + |E|)$, its highly serialized iterative peeling process must be fundamentally restructured to leverage modern multi-core and distributed processing architectures¹⁵. To avoid extreme I/O blocking and cache latency on massive networks, an improved CWIIF implementation must adopt an asynchronous, vertex-centric parallel K-shell approach¹⁴.

In advanced parallel models—such as those utilizing dependency Directed Acyclic Graphs (DAGs) and Local Degree Subgraphs (LDS)—vertices independently estimate and bound their own coreness levels¹⁴. During each computational batch, vertices dynamically update their internal states by polling the degrees of their immediate neighbors and migrating between core-level buckets via atomic memory operations. This approach entirely eliminates the need for strict global synchronization after every single node deletion¹⁴. By decoupling the vertex updates, the decomposition can utilize the full instruction width of distributed compute clusters or modern GPU architectures, reducing practical execution time by orders of magnitude¹⁴. Additionally, for datasets that drastically exceed RAM limits, algorithms specifically engineered for external-memory processing (e.g., EMcore) must be deployed to allow sub-graph processing directly from secondary storage without incurring fatal disk fragmentation¹⁵.

2. Generalizing to Weighted and Directed Multilayer Networks

To transcend the baseline limitations regarding unweighted and undirected graphs, the fundamental metrics operating within the CWIIF framework must be mathematically generalized.

Directed Pathing and Flow Routing: For directed graphs, the definitions of active paths and betweenness centrality must accurately respect edge orientation, rendering the adjacency matrix asymmetric. The adaptive sampling mechanisms used for betweenness centrality naturally extend to directed graphs by executing targeted, directed BFS or directed Dijkstra traversals²⁷. The K-shell decomposition process must also be modified to utilize either strict *in-degree*, strict *out-degree*, or a combinatorial Mixed Degree Decomposition (MDD) that assesses the influence of both remaining and removed neighbors during the parallel peeling process¹⁷. When the objective is influence maximization—triggering an outward

cascade—out-degree coreness proves structurally more relevant than generic undirected coreness.

Handling Varied Edge Weights: In systems featuring weighted edges, traditional discrete K-shell decomposition immediately fails. CWIIF 2.0 must implement generalized weighted K-shell formulations, where a node's topological degree is dynamically calculated as the sum of its adjacent edge weights or as the mathematical product of its discrete degree and the average weight of its incident links⁷. For the NAP and BC calculations, unweighted BFS traversals must be systematically replaced by Dijkstra's algorithm, utilizing optimized priority queues (e.g., Fibonacci heaps) to compute shortest paths based on actual connection costs¹¹. Crucially, the implementation must account for the semantic meaning of the edge weights. If edge weights denote capacities or probabilities (where a higher value is superior, such as an infection probability λ in the SIR model), the weight values must be inverted via

transformations such as $w'_{ij} = \frac{1}{w_{ij}}$ or $w'_{ij} = -\log(w_{ij})$ prior to executing Dijkstra's algorithm¹. This mathematical inversion guarantees that shortest path algorithms correctly identify the routes of maximum likelihood or maximum flow, rather than simply paths with the lowest numerical weight values.

3. Advanced Inter-layer Coupling: Beyond Jaccard Intersection

The baseline algorithm's reliance on the Jaccard coefficient to compute the Normalized Inter-layer Communication Intersection (NICI) is topologically primitive. A basic intersection of edge lists completely fails to capture the higher-order topological features—such as structural triangles, network motifs, and overlapping community boundaries—that dictate how dynamic processes actually leak from one layer to another³⁰.

Tensor-based Formulations and Hodge Laplacians: To accurately quantify true inter-layer communication and structural coupling, CWIIF 2.0 should mathematically represent the multilayer network using a fourth-order adjacency tensor $\mathcal{A} \in \mathbb{R}^{N \times N \times L \times L}$ ³². Rather than isolating layers and analyzing basic edge overlap, the system must evaluate inter-layer dependencies through rigorous spectral analysis of the supra-adjacency matrix or supra-Laplacian³.

A substantially more robust equivalent to NICI can be derived by calculating cross-layer topological consistency using generalized Hodge Laplacians³⁰. By evaluating the block incidence operator across all interconnected layers, the algorithm can quantify not just the strict binary overlap of 1-simplices (edges), but the deep structural consistency of 2-simplices

(triangles) and higher-order network cavities. Defining the lower Hodge Laplacian for layer ℓ as $\mathcal{L}_{k,\downarrow}^{[\ell]} = (\mathcal{B}_k^{[\ell]})^\top \mathcal{W}_{k-1}^{[\ell]} \mathcal{B}_k^{[\ell]}$, the total cross-layer topological coupling can be calculated through persistent homology descriptors³⁰. This advanced mathematical approach measures the exact diffusion permeability and structural resonance between layers, natively handling sparse networks where exact 1:1 edge overlap is rare but overarching topological community similarities are highly pronounced³⁰.

Baseline Component	Enhanced Implementation Strategy	Direct Topological Benefit
Unweighted SSSP	Priority-Queue Dijkstra with Weight Inversion	Identifies paths of maximum capacity/probability, not just fewest hops.
Undirected Degrees	MDD / Out-Degree Focus	Prevents network sinks from being falsely identified as influence spreaders.
Jaccard Intersection	Fourth-Order Tensors & Hodge Laplacians ³⁰	Captures higher-order motif consistency and cross-layer diffusion permeability.

Dynamic Weighting and Evolutionary Seed Selection

The most urgent methodological correction required to modernize CWIIF is discarding the rigid equal-weighting algorithms and the destructive linear scalarization processes that dictate final seed selection.

The Entropy Weight Method (EWM) for Objective Factorization

To entirely eliminate arbitrary heuristic biases, CWIIF 2.0 must dynamically determine the weights of the NAN, NAP, and NICI factors algorithmically, basing the decision on the actual statistical variance of the target network. The Entropy Weight Method (EWM) serves as the optimal mathematical solution for this requirement³⁴. EWM calculates strictly objective criterion weights by measuring the information entropy—the degree of structural uncertainty—of each individual factor across all network layers.

If a specific factor (e.g., NAN) remains relatively uniform across all layers within a given dataset (meaning every layer possesses a nearly identical proportion of active nodes), its calculated information entropy will be exceedingly high. Consequently, EWM will assign it a low coefficient weight, correctly identifying that the factor lacks the discriminative power necessary to differentiate layer importance³⁴. Conversely, if the NAP factor varies wildly from layer to layer—indicating that certain layers possess massive, unique topological advantages in transmission efficiency—its calculated entropy will be low, prompting EWM to assign it a mathematically dominant weight³⁴.

By defining the objective weight of factor j as $\omega_j = \frac{1-E_j}{\sum(1-E_j)}$ (where E_j represents the normalized entropy of the specific factor), the computation of the layer-weighted coefficient

becomes a fully dynamic, data-driven calculation:

$$W(\alpha) = \omega_1 N A N^{[\alpha]} + \omega_2 N A P^{[\alpha]} + \omega_3 N I C I^{[\alpha]}$$

This completely resolves the rigidity of the baseline formulation, allowing the algorithm to seamlessly adapt to the unique physics of any input dataset, whether it be a genetic expression network or an urban transit grid. This exact entropy-based adaptive weighting logic should also be applied when integrating KS and BC within the intra-layer calculations, dynamically scaling the relevance of local coreness versus global bridging capacity based on the unique spatial geometry of the individual layer.

Nonlinear Vector Integration via Memetic Algorithms

To effectively address the dimensionality collapse occurring in the final algorithmic step (

$T I_i = \sum K B_i^{[\alpha]} \times W(\alpha)$), CWIIF 2.0 must defer scalarization until the exact moment of influence seed selection. Instead of blindly ranking nodes by a collapsed, one-dimensional scalar, the optimized algorithm must retain the fully intact L -dimensional vector $\mathbf{KB}_i$ for each node throughout the entire process.

When executing Influence Maximization (IM), identifying the absolute top K influential nodes must account for overlapping spheres of influence²¹. Selecting multiple nodes that exert their primary influence within the exact same network layer leads to massive redundancy and squandered selection budgets. Therefore, CWIIF 2.0 should employ Memetic Algorithms (MAs) or Multi-objective Evolutionary Algorithms (MOEAs) that operate directly upon the preserved multilayer vector space².

By deploying a Multilayer Multi-population Genetic Algorithm (MMGA), vast populations of potential seed sets can be iteratively evolved and tested²². The specific fitness function utilized to evaluate a candidate seed set does not rely on a simple scalar sum; rather, it simulates the aggregate influence spread across the full tensor representation of the network²². Evolutionary

operators—such as crossover and mutation algorithms—utilize the preserved $\mathbf{KB}_i$ influence vectors as heuristic probabilities to bias the initial selection of seed genes²². However, the overarching evolutionary optimization process explicitly minimizes the influence overlap penalty while maximizing multi-scenario structural robustness across diverse network layers²¹. This advanced approach guarantees that the final output set of influential nodes is globally dispersed, maximizing systemic coverage across deeply varied subgraphs and operational layers—a sophisticated capability that remains mathematically impossible within the rigid confines of the baseline CWIIF's dot-product scalarization paradigm.

Works cited

1. CWIIF.pdf

2. IDENTIFYING INFLUENTIAL SPREADERS IN COMPLEX NETWORKS,
<https://scholarworks.iu.edu/iuswrrest/api/core/bitstreams/fde8bce2-6054-46e4-979a-6a934df0b188/content>
3. INFUSE - World Scientific Publishing,
<https://www.worldscientific.com/doi/pdf/10.1142/S0129183127501427?download=true>
- 4.
5. Identifying important nodes in complex networks based on extended,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9950367/>
6. Key Node Identification in Transportation Networks Based on the K,
<https://www.mdpi.com/2079-8954/14/8/943>
7. Core Decomposition in Graphs: Concepts, Algorithms and,
<https://openproceedings.org/2016/conf/edbt/paper-d.pdf>
8. Learning to Identify High Betweenness Centrality Nodes from Scratch,
https://web.cs.ucla.edu/~yzsun/papers/2019_CIKM_Betweenness.pdf
9. Betweenness centrality - Wikipedia,
https://en.wikipedia.org/wiki/Betweenness_centrality
10. Betweenness Centrality Approximation in Large Networks using,
<https://ebooks.spiedigitallibrary.org/proceedings/Download?urlId=10.1117%2F12.3013414>
11. A Faster Parallel Algorithm and Efficient Multithreaded ... - OSTI.GOV,
<https://www.osti.gov/servlets/purl/951102>
12. Better Approximation of Betweenness Centrality,
<https://turing.iem.thm.de/routeplanning/hwy/betweennessFull.pdf>
13. Fast Approximation of Centrality - UC Irvine,
<https://ics.uci.edu/~eppstein/pubs/EppWan-SODA-01.pdf>
14. Parallel k-Core Decomposition with Batched Updates ... - Julian Shun,
<https://jshun.csail.mit.edu/async-kcore.pdf>
15. Detecting and Exploring Homogeneous Dense Groups via k-Core,
<https://www.mdpi.com/2076-3417/15/19/10753>
16. xiaoke Xu - ORCID, <https://orcid.org/0000-0002-9148-3145>
17. A node influence ranking algorithm combining k-shell iteration and,
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0328381>
18. Centrality in Networks: Finding the Most Important Nodes,
https://webs-deim.urv.cat/~sergio.gomez/papers/Gomez-Centrality_in_networks-Finding_the_most_important_nodes.pdf
19. Zhanwei DU - ORCID, <https://orcid.org/0000-0002-2020-767X>
20. International Journal of Modern Physics C - World Scientific Publishing,
<https://www.worldscientific.com/doi/10.1142/S0129183127501427>
21. Influence maximization in multilayer networks based on adaptive,
<https://www.semanticscholar.org/paper/Influence-maximization-in-multilayer-networks-based-Zhang-Xie/e61f287e3c88d7e235d6c421fede4c2423627406>
22. BCIM: Budget and capacity constrained influence maximization in,
<https://arxiv.org/pdf/2410.04820>
23. A Learning Theory Approach to Ranking Nodes in Large Networks,

- <https://par.nsf.gov/servlets/purl/10356172>
24. Fast Approximation of Centrality - EMIS,
<http://emis.icm.edu.pl/journals/JGAA/accepted/2004/EppsteinWang2004.8.1.pdf>
 25. View of Fast Approximation of Centrality,
<https://jgaa.info/index.php/jgaa/article/view/paper81/2883>
 26. K-hop centrality metric for identifying influential spreaders in,
<https://www.semanticscholar.org/paper/4ffa59a8a1ddf088e20d1230f32d76450c1c3e1e>
 27. Fast Shortest Path Distance Estimation in Large Networks,
https://francescobonchi.com/paper_7.pdf
 28. Node Centrality in Weighted Networks | Tore Opsahl,
<https://toreopsahl.com/tnet/weighted-networks/node-centrality/>
 29. Edge Sparsification for Centrality Computation - Christian Schulz,
https://schulzchristian.github.io/thesis/ma_litzinger.pdf
 30. Algebraic Topology Modeling and Game Decision Optimization for,
<https://www.mdpi.com/2227-7390/14/11/1817>
 31. Key Node Identification Method Based on Multilayer Neighbor Node,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11727151/>
 32. Effectively Handling New Relationship Formations in Closeness,
<http://cs.iit.edu/~jkorah/papers/C-ATAW-16.pdf>
 33. Multilayer Network Centrality Explained | PDF | Applied Mathematics,
<https://www.scribd.com/document/889913927/Group-c-1>
 34. Entropy-Weight-Method-Based Integrated Models for Short-Term,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9317321/>
 35. Integrating centrality measures and multi-criteria decision-making for,
<https://www.frontiersin.org/journals/sustainable-food-systems/articles/10.3389/fsufs.2026.1731735/full>
 36. A multi-factorial evolutionary algorithm concerning diversity,
<https://www.tandfonline.com/doi/full/10.1080/09540091.2023.2275534>